OTUS

ОНЛАЙН-ОБРАЗОВАНИЕ

Разработчик C++

Базовый курс

Полиморфизм

Сергей Кольцов
профессиональный программист

Что будем делать

- ещё раз про функции
- ещё раз про структуры
- ещё раз про классы
- ~~ещё раз~~ про полиморфизм

Функции

Функции - это

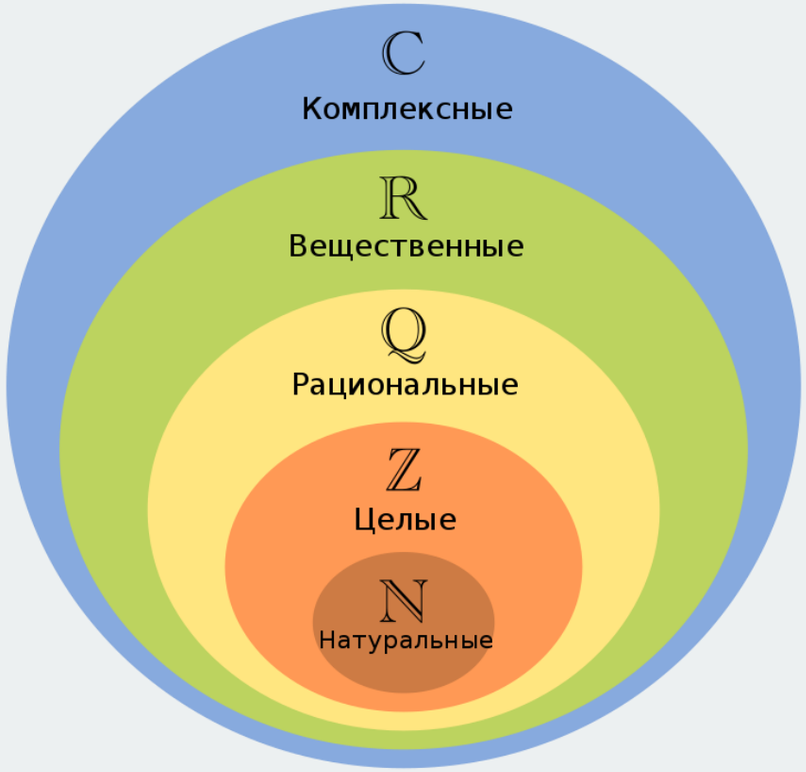
- удобно
- просто
- мощно
- … но довольно низкоуровнево

Комплексные числа

a + b*i

где a и b - вещественные,

i - мнимая единица (sqrt(-1))

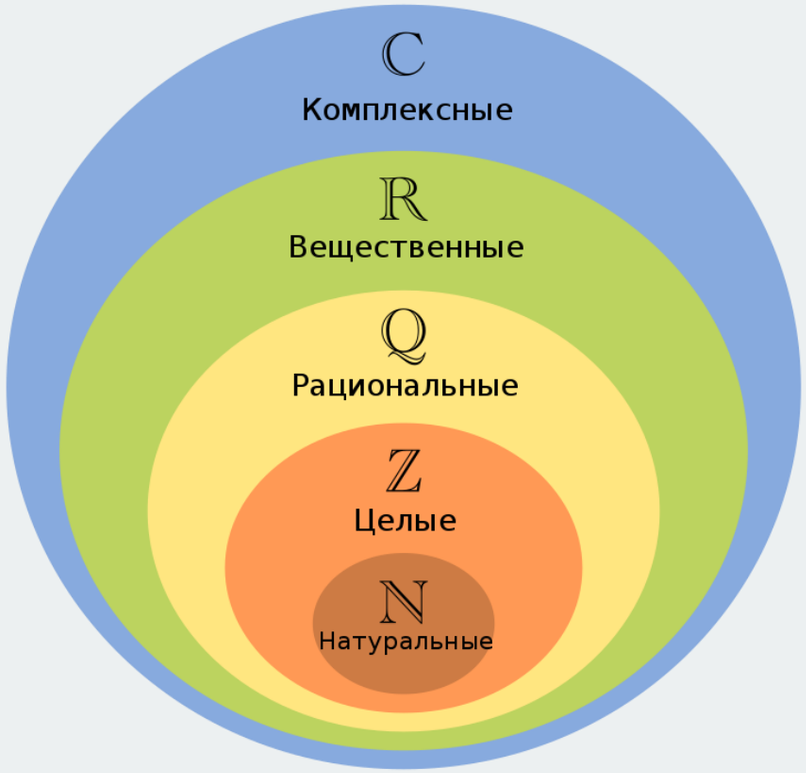


Комплексные числа

a + b*i

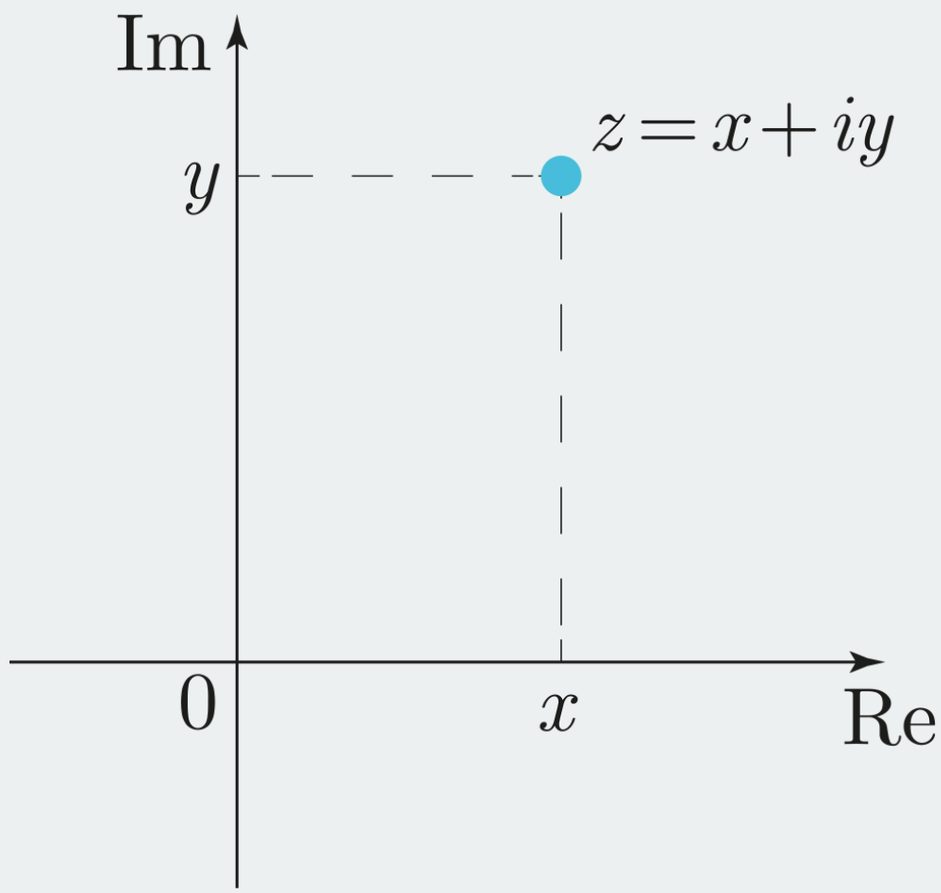
где a и b - вещественные,

i - мнимая единица (sqrt(-1))

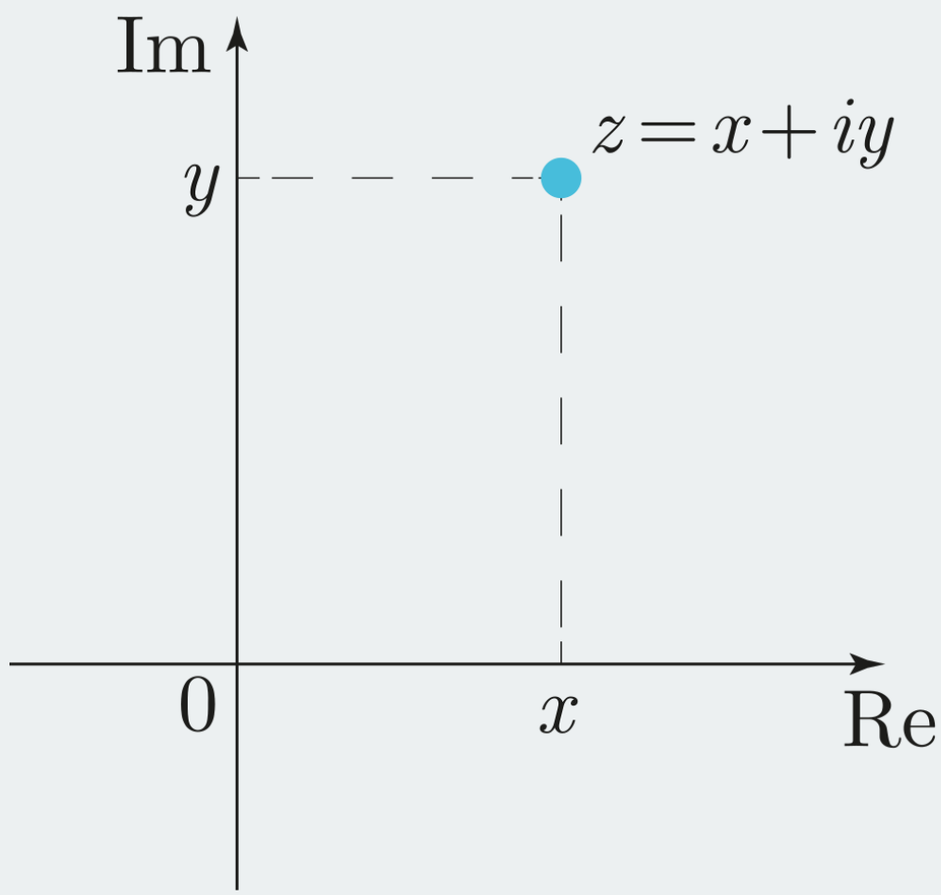


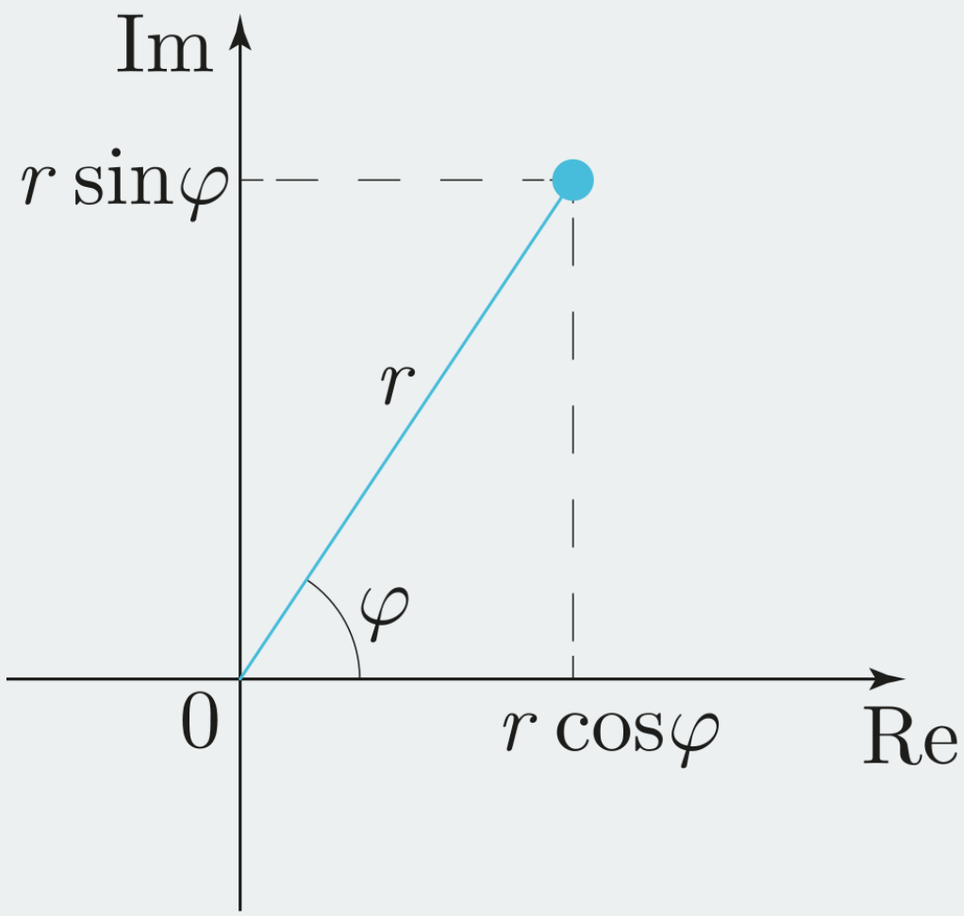
- удобная абстракция
- для решения многих задач
- хотя поначалу немного выносит мозг…

Комплексные числа



Комплексные числа





Функции

Функции - это

- удобно
- просто
- мощно
- … но довольно низкоуровнево
- … и состояние приходится хранить вовне

Функции

Функции - это

- удобно
- просто
- мощно
- … но довольно низкоуровнево
- … и состояние приходится хранить вовне

Тем не менее, даже так можно написать сложные приложения

Функции

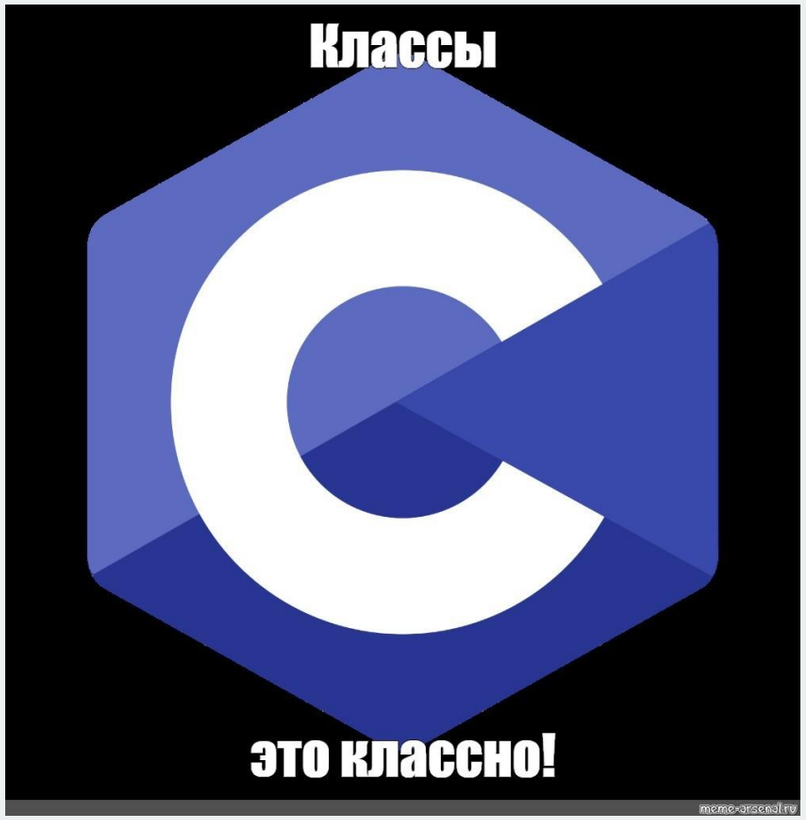
Функции - это

- удобно
- просто
- мощно
- … но довольно низкоуровнево
- … и состояние приходится хранить вовне

Тем не менее, даже так можно написать сложные приложения

Linux написан исключительно на функциях и простых структурах (язык Си)

Классы



Классы - это

- ещё более удобно
- куда менее просто
- но куда более мощно
- почти максимальный уровень абстракции (больше только метапрограммирование, но о нём позже)
- позволяют хранить состояние

ООП

ООП - это

- абстрагирование
- инкапсуляция
- наследование
- полиморфизм

ООП

ООП - это

- абстрагирование
- инкапсуляция
- наследование
- полиморфизм

А всё это зачем?

ООП

ООП - это

- абстрагирование
- инкапсуляция
- наследование
- полиморфизм

А всё это зачем?

Да просто для ускорения и удешевления процесса разработки сложных продуктов.

ООП

ООП

- выстраиваем взаимодействие
- между объектами разных классов
- тем самым делая упор на бизнес-логике
- что является естественным походом

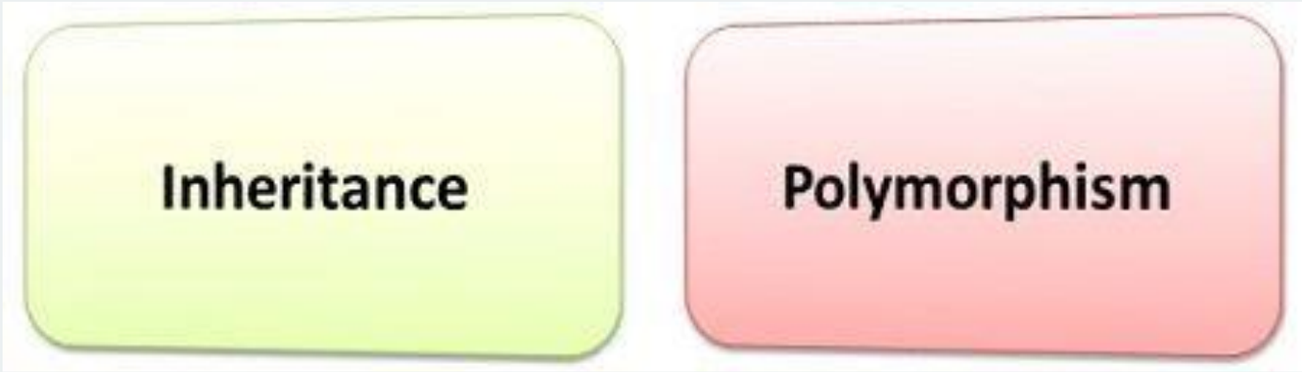
ООП

ООП

- выстраиваем взаимодействие
- между объектами разных классов
- тем самым делая упор на бизнес-логике
- что является естественным походом

Хотя кое-кто считает более естественным подход, когда упор делается на поток обработки данных (функциональная парадигма)

Полиморфизм



Полиморфизм:

- реализуется через наследование
- но смысл уже не только в переиспользовании
- а в возможности изменения поведения
- без раскрытия дополнительной информации

Код

Объявляем базовый класс:

```
1.  class Felidae {
2.  public:
3.      Felidae(const std::string& name)
4.          : m_name{name}
5.      {}
6.
7.      virtual void speak() const = 0;
8.
9.      virtual void call() const {
10.         std::cout << m_name << ", come here!" << std::endl;
11.     }
12. private:
13.     std::string m_name;
14. };
```

Код

Объявляем базовый класс:

```
1.  class Felidae {
2.  public:
3.      Felidae(const std::string& name)
4.          : m_name{name}
5.      {}
6.
7.      virtual void speak() const = 0;
8.
9.      virtual void call() const {
10.         std::cout << m_name << ", come here!" << std::endl;
11.     }
12. private:
13.     std::string m_name;
14. };
```

Это что за ~~покемон~~ новый синтаксис?

Код

Объявляем базовый класс:

```
1.  class Felidae {
2.  public:
3.      Felidae(const std::string& name)
4.          : m_name{name}
5.
6.
7.      virtual void speak() const = 0;
8.
9.      virtual void call() const {
10.         std::cout << m_name << ", come here!" << std::endl;
11.     }
12. private:
13.     std::string m_name;
14. };
```

Новое ключевое слово

Это что за ~~покемон~~ новый синтаксис?

Как это выглядит в коде.

Код

Виртуальные методы:

- меняют поведение наследника
- с полным сохранением сигнатуры
- позволяют работать с наследником по указателю
- или ссылке на базовый класс
- что и называется «полиморфизм»
- и даёт возможность абстрагироваться от деталей реализации

Код

В наследнике реализуем:

```
1.  class Cat : public Felidae {
2.  public:
3.      Cat(const std::string& name)
4.          : Felidae{name} {}
5.
6.      virtual void speak() const {
7.          std::cout << "Meow, meow!" << std::endl;
8.      }
9.      virtual void call() const {
10.         std::cout << m_name << ", kitty, kitty, kitty!";
11.     }
12. };
```

Спасибо за внимание!

Заполните, пожалуйста опрос о занятии.

Ответы на вопросы